Buzz Simple Demo: Web Page + n8n

Full build guide. BongoHive AI Lab. Goal: a working talk-to-Buzz demo in about 1 to 2 hours, with no GPU and no phone system. Use **fake data only**.

1. What you will build

A web page with a talk button. You speak, the browser turns speech into text, sends it to n8n, an AI model writes the reply, and the browser reads it aloud. A typed-text box is the backup if the microphone fails during the demo.

```
Browser (Chrome)  --text-->  n8n Webhook  -->  AI Agent (LLM + memory)
   ^  mic + speaker                                  |
   +-------------- {reply} <-- Respond to Webhook <------+
                               (optional) --> Google Sheet log
```

This is a **demo path**, separate from the full Buzz platform in docs/PLAN.md (Dograh, vLLM, Speaches, Asterisk). It proves the idea, the script and the n8n flow. Later, the same n8n workflow can receive call summaries from the real platform.

2. What you need

Item	Choice	Notes
n8n	n8n Cloud trial (fastest) **or** Docker on a small cloud VM	Cloud trial needs no server. Self-host needs HTTPS (section 3B).
AI model	An API key: OpenAI, Anthropic or Google	Cloud API. Fine for fake test data; this is not the final local setup.
Browser	Google Chrome or Edge	Firefox lacks speech recognition. Safari is unreliable.
Page hosting	Your laptop (localhost) or any static host	Microphone only works on HTTPS or localhost.
Files	demo/index.html (in this repo)	Full code is in section 5.

3. Step 1: Get n8n running

Option A: n8n Cloud (recommended for the demo)

• Create an account at n8n.io and start the free trial. You get an https address like *yourname.app.n8n.cloud*.

• Skip to Step 2.

Option B: self-host on a cloud VM

• Ubuntu VM, 2 vCPU and 4 GB RAM, Docker installed, a domain pointing to its IP (for example n8n-demo.bongohive.co.zm).

• Open ports 80 and 443 only.

```
docker volume create n8n_data
docker run -d --name n8n --restart unless-stopped -p 127.0.0.1:5678:5678 \
  -e N8N_HOST=n8n-demo.example.com -e N8N_PROTOCOL=https \
  -e WEBHOOK_URL=https://n8n-demo.example.com/ \
  -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n

# HTTPS in front (Caddy):
sudo apt install -y caddy
echo 'n8n-demo.example.com { reverse_proxy 127.0.0.1:5678 }' | sudo tee /etc/caddy/Caddyfile
sudo systemctl reload caddy
```

Open https://n8n-demo.example.com and create the owner account. Replace the example domain with yours.

4. Step 2: Build the n8n workflow

In n8n click **Create workflow**, name it **Buzz Demo**, then add four nodes in this order.

#	Node	Settings
1	**Webhook**	HTTP Method: POST. Path: *buzz-demo*. Respond: **Using 'Respond to Webhook' Node**. Options, Allowed Origins (CORS): * (for the demo).
2	**AI Agent**	Source for Prompt: Define below. Prompt: *{{ $json.body.message }}*. Options, System Message: see below.
2a	Chat Model (sub-node of the agent)	OpenAI Chat Model (or Anthropic or Google). Select your credential and a fast, small model.
2b	Simple Memory (sub-node)	Session ID: Define below, *{{ $('Webhook').item.json.body.sessionId }}*. Context window: 8.
3	**Respond to Webhook**	Respond With: JSON. Response Body: *{{ { reply: $json.output } }}* (expression mode).

System message (paste into the agent)

```
You are Buzz, the friendly voice assistant of BongoHive, a technology hub
in Lusaka, Zambia. You are speaking, not writing: reply in 1 to 3 short
sentences, plain words, no lists, no emojis, no markdown.
Answer only about BongoHive: programmes, events, coworking, contact.
If you do not know, say so and offer to take the person's name and number.
Never ask for PINs, bank cards or ID numbers.
```

Add your own facts (opening hours, programme names, contact details) under the system message as a short **Facts:** list. A 10-line fact list is enough for a convincing demo and avoids made-up answers.

Test and publish

• Click **Execute workflow** on the canvas, then in a terminal run the curl test below using the **Test URL** shown in the Webhook node. You should get {"reply": "..."}.

• Toggle **Active** (top right). From now on use the **Production URL** (it contains /webhook/ and not /webhook-test/). This is the most common mistake.

```
curl -X POST "https://YOUR-N8N/webhook/buzz-demo" \
  -H "Content-Type: application/json" \
  -d '{"message":"What does BongoHive do?","sessionId":"t1"}'
```

Optional extra: log every question

After the Webhook node, add a **Google Sheets: Append Row** node (columns: time, sessionId, message) in parallel to the agent. It gives you a live log to show during the demo. Do not log real personal data.

5. Step 3: The web page

The file is already saved at **demo/index.html**. Open it and replace *PASTE_YOUR_N8N_PRODUCTION_WEBHOOK_URL_HERE* with your Production URL. Full code:

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Buzz Voice Demo</title>
<style>
  body{font-family:system-ui,sans-serif;max-width:520px;margin:0 auto;padding:16px;background:#fafafa;color:#111}
  h1{font-size:1.3rem} #log{height:50vh;overflow:auto;background:#fff;border:1px solid #ddd;border-radius:8px;padding:10px}
  .u{color:#555;margin:6px 0} .b{font-weight:600;margin:6px 0}
  button{padding:12px 18px;border:0;border-radius:8px;background:#111;color:#fff;font-size:1rem;margin:8px 4px 0 0}
  button.on{background:#c00} input{width:70%;padding:10px;border:1px solid #ccc;border-radius:8px}
</style>
</head>
<body>
<h1>Buzz - BongoHive AI Lab (demo)</h1>
<div id="log"></div>
<button id="mic">Hold to talk</button>
<div style="margin-top:8px"><input id="txt" placeholder="...or type here"><button id="send">Send</button></div>
<script>
const WEBHOOK_URL = "PASTE_YOUR_N8N_PRODUCTION_WEBHOOK_URL_HERE";
const sid = "demo-" + Math.random().toString(36).slice(2);
const log = document.getElementById("log"), mic = document.getElementById("mic");
const add = (cls, t) => { const d = document.createElement("div"); d.className = cls; d.textContent = t; log.append(d); log.scrollTop = log.scrollHeight; };

async function ask(message) {
  add("u", "You: " + message);
  try {
    const r = await fetch(WEBHOOK_URL, { method: "POST", headers: {"Content-Type": "application/json"},
      body: JSON.stringify({ message, sessionId: sid }) });
    const { reply } = await r.json();
    add("b", "Buzz: " + reply);
    speechSynthesis.cancel();
    speechSynthesis.speak(new SpeechSynthesisUtterance(reply));
  } catch (e) { add("b", "Error: could not reach the server."); }
}

const SR = window.SpeechRecognition || window.webkitSpeechRecognition;
if (SR) {
  const rec = new SR(); rec.lang = "en-US"; rec.interimResults = false;
  rec.onresult = e => ask(e.results[0][0].transcript);
  rec.onend = () => mic.classList.remove("on");
  const start = () => { speechSynthesis.cancel(); mic.classList.add("on"); rec.start(); };
  mic.onmousedown = mic.ontouchstart = start;
  mic.onmouseup = mic.ontouchend = () => rec.stop();
} else { mic.disabled = true; mic.textContent = "Use Chrome for voice"; }

document.getElementById("send").onclick = () => { const t = txt.value.trim(); if (t) { txt.value = ""; ask(t); } };
txt.onkeydown = e => { if (e.key === "Enter") send.click(); };
add("b", "Buzz: Hello! I'm Buzz from BongoHive. How can I help?");
</script>
</body>
</html>
```

Run it

```
cd voice-agent/demo
python3 -m http.server 8080
# open  http://localhost:8080  in Chrome, allow the microphone
```

To share a link with others, upload the single file to any HTTPS static host (Netlify Drop, GitHub Pages or an nginx folder on your cloud VM).

6. Demo-day checklist

• Workflow is **Active** and the page uses the **Production URL**.

• Run 5 test questions the night before; fix wrong answers by editing the Facts list.

• Use Chrome, headphones or a good speaker, and a quiet room. Check the microphone permission.

• Have the typed-text box ready as the fallback and keep the n8n execution list open to show what happens behind the scenes.

• Plan the script: greeting, 3 normal questions, 1 question Buzz cannot answer (shows the safe fallback), 1 request to be called back.

• Internet check: the demo needs the network for speech recognition, n8n and the AI model.

• Use only fake or public information. No real customer data on this cloud setup.

7. Troubleshooting

Problem	Fix
Page says error / nothing returned	Wrong URL (test vs production), workflow not Active, or CORS not set on the Webhook node. Check the browser console (F12).
Mic button does nothing	Not Chrome, page not on HTTPS or localhost, or microphone blocked in the site settings.
Reply is a long essay	Tighten the system message: 'maximum 2 sentences'.
No voice reply	Click anywhere on the page first (browsers block audio until you interact), and check the volume.
Slow answers (over 4 s)	Choose a smaller, faster model in the Chat Model node.

8. Making it ready for real use (later)

• **Security:** restrict CORS to your page's domain, not *. Add a secret header check and rate limiting at nginx or Caddy.

• **Privacy:** move to the local servers; replace the cloud AI with the local model (see docs/PLAN.md).

• **Voice quality:** browser speech is basic. The Buzz platform (Dograh with Speaches and Kokoro) gives better voices and phone calls.

• **Business hookup:** add HubSpot nodes to save a lead's name and number, only with consent.

• **Migration:** n8n workflows export as JSON, so they move to a local n8n unchanged.